

Sistema web de gestión y control de citas para una barbería

Diseño, desarrollo, implementación y evaluación del prototipo

Alumno

Fecha .. de septiembre de 2026

Propósito del documento

En esta segunda parte retomo lo que propuse para la barbería y explico cómo quedó el sistema. Ya hay una aplicación que permite reservar y administrar citas. Aquí presento sus pantallas, la forma en que guarda los datos y las pruebas que respaldan su funcionamiento.

Resultado principal

El sistema ya permite hacer una reserva, cambiarla de horario y consultar qué pasó con ella. También tiene la configuración de jornadas y bloqueos. En la prueba local pasaron los 26 casos. En el registro aparecen 27 resultados porque se cuenta además el grupo de integración completo, pero no son 27 pruebas distintas.

Índice e introducción

Contenido del informe	Página
Portada y propósito	1
Índice e introducción	2
Diseño de la arquitectura	3
Diseño de procesos y reglas	4
Diseño de la base de datos	5
Diseño de la interfaz	6
Desarrollo e implementación	7
Pruebas y resultados	8
Evaluación de calidad	9
Conclusiones y referencias	10

Introducción

En la primera entrega planteé el caso de dos clientes que piden la misma hora. El problema parecía sencillo, hasta considerar que cada servicio dura un tiempo distinto y que el barbero también tiene descansos. Para esta segunda parte mantuve esa misma preocupación: que la agenda no ofrezca un espacio donde en realidad no se puede atender.

La propuesta ya funciona como una aplicación web. El cliente entra, revisa los servicios y elige un horario sin tener que crear una cuenta. El encargado tiene otro acceso, donde sí necesita iniciar sesión. Desde ahí puede revisar las citas del día, cambiar un horario o resolver una cancelación.

Lo que me interesa comprobar aquí es qué pasa al guardar. Dos personas todavía pueden ver la misma hora libre, porque una puede estar llenando sus datos mientras la otra confirma. La revisión del servidor tiene que resolver ese caso. Una pantalla que muestre bien los horarios no alcanza por sí sola.

Alcance y condiciones

Conservé el alcance de la primera parte. El sistema maneja servicios, barberos, clientes, citas y horarios, con descansos y bloqueos cuando hagan falta. Dejé fuera pagos en línea, inventario, nómina y recordatorios automáticos. Tampoco hay una cuenta para cada barbero. Los horarios de la demostración sirven para probar el proyecto; todavía tengo que acordar esas reglas con un encargado real.

Diseño de la arquitectura

Separación de responsabilidades

Mantuve la separación que propuse al principio. React muestra los formularios y la agenda en el navegador. Cuando alguien confirma una cita, la solicitud pasa por la API y llega al servidor con Node.js y Express. Ahí se revisa si puede guardarse en MySQL. Así, el navegador no entra directamente a la base de datos ni decide por su cuenta que una reserva es válida.

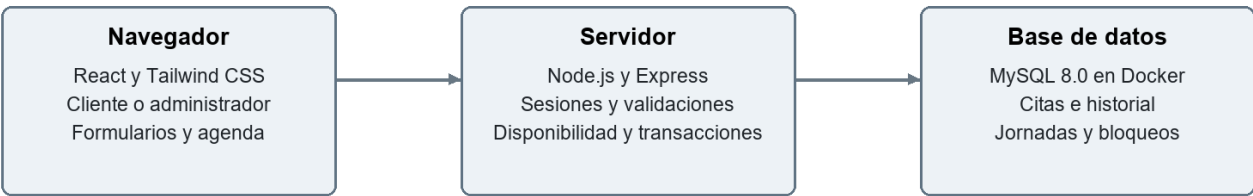


Figura 1. Comunicación entre la pantalla, el servidor y la base de datos. La interfaz intercambia datos por HTTP en formato JSON. El servidor usa consultas SQL con parámetros.

Componente	Responsabilidad	Elementos principales
Interfaz	Capturar datos y mostrar respuestas	Reserva, agenda, catálogos, jornadas, bloqueos y clientes
API	Autorizar y validar operaciones	Sesiones, rutas públicas y administrativas, reglas de disponibilidad
Persistencia	Guardar relaciones e historial	MySQL, claves foráneas, transacciones e índices

Organización de los módulos

Organicé el servidor por tareas para poder revisar una parte sin tener que buscar en todo el código. Hay módulos para el acceso, las consultas y las reglas de la agenda, entre otros. En React, cada apartado del administrador tiene su componente. Algunas funciones se comparten, como enviar solicitudes o mostrar un mensaje. Si cambia una de esas funciones, no hace falta corregirla pantalla por pantalla.

Acceso y comunicación

El cliente puede ver los servicios y los horarios libres, pero no necesita conocer quién reservó los demás espacios. Por eso, los nombres y teléfonos quedan en la parte administrativa. Para entrar ahí hace falta una sesión vigente. Las contraseñas se procesan con `bcrypt` y una sal aleatoria, y la cookie de sesión usa `HttpOnly` y `SameSite Strict`. Conservo estos nombres porque son las opciones concretas que usa el código. La referencia [3] explica `bcrypt`.

Para mostrar el proyecto, la interfaz y la API funcionan en la misma dirección local. Durante el desarrollo, Vite redirige las solicitudes al servidor. Esto deja trabajar en las pantallas sin mover las reglas que revisan las citas.

Diseño de procesos y reglas

Ver una hora libre no significa que ya esté apartada. El cliente aún tiene que confirmar. En ese momento, el servidor vuelve a consultar la disponibilidad y guarda la cita dentro de una transacción. Si algo falla, revierte la operación para no dejar el cambio a medias.

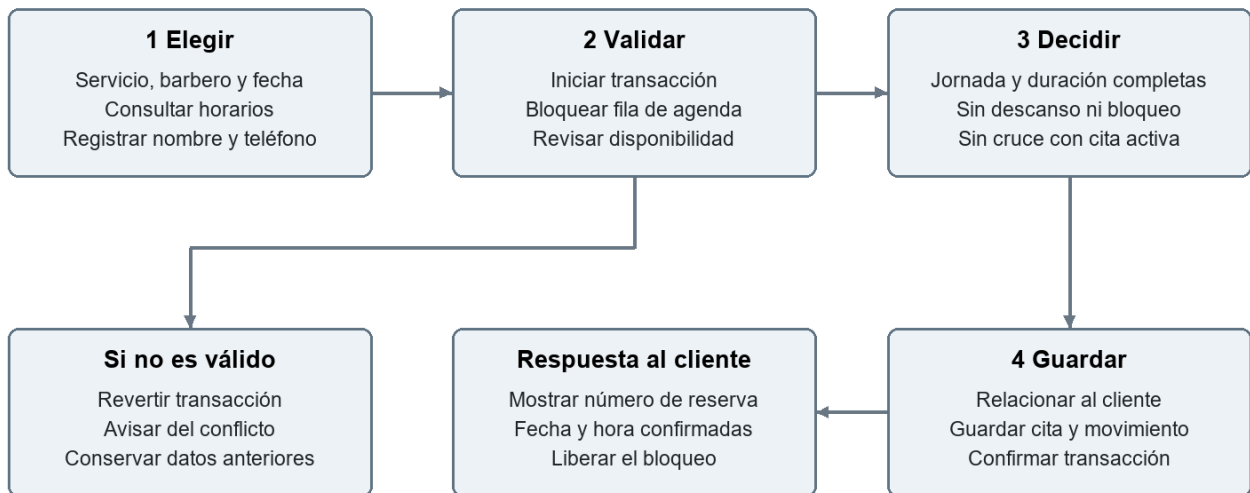


Figura 2. Pasos para reservar una cita. Si la validación falla, el proceso sigue la ruta inferior izquierda y devuelve un aviso al cliente.

Regla de conflicto

La regla sigue siendo la de la primera entrega: hay cruce si $\text{inicio_nueva} < \text{fin_existente}$ y $\text{fin_nueva} > \text{inicio_existente}$, para el mismo barbero y una cita pendiente o confirmada. Por ejemplo, si una cita termina a las 10:40, otra sí puede empezar a las 10:40. Lo que no puede hacer es empezar antes y ocupar parte del servicio anterior. También debe caber completa en la jornada, fuera de descansos y bloqueos.

Control de solicitudes simultáneas

Para atender las solicitudes que llegan al mismo tiempo, el servidor usa SELECT FOR UPDATE sobre una fila de control [2]. En términos sencillos, una operación revisa y guarda mientras las demás esperan. Cuando les toca continuar, consultan otra vez si el horario está libre. El bloqueo termina al confirmar o revertir la transacción. Esta solución permite controlar los cruces, aunque con mucha más demanda esa espera podría convertirse en una limitación.

Cambios y estados

También cuidé el cambio de horario. Primero se revisa el espacio nuevo y, si no sirve, la cita se queda como estaba. Al cancelar, el horario vuelve a estar libre y el registro sigue en el historial. Se puede marcar una cita como completada después de su fin, o como ausencia después de su inicio. Hay un ajuste respecto a lo planeado: la reserva pública queda confirmada directamente. El estado pendiente existe en el modelo y admite cambios, pero el formulario público no lo crea.

Diseño de la base de datos

Cada cita necesita relacionarse con el servicio y el barbero que la atenderá. Los registros nuevos también quedan ligados a un cliente. Para conservar esas relaciones uso claves foráneas. Si un servicio o un barbero deja de estar disponible, se marca como inactivo; borrarlo haría más difícil conservar sus citas anteriores. Las tablas de acceso están separadas de los catálogos y de las citas.

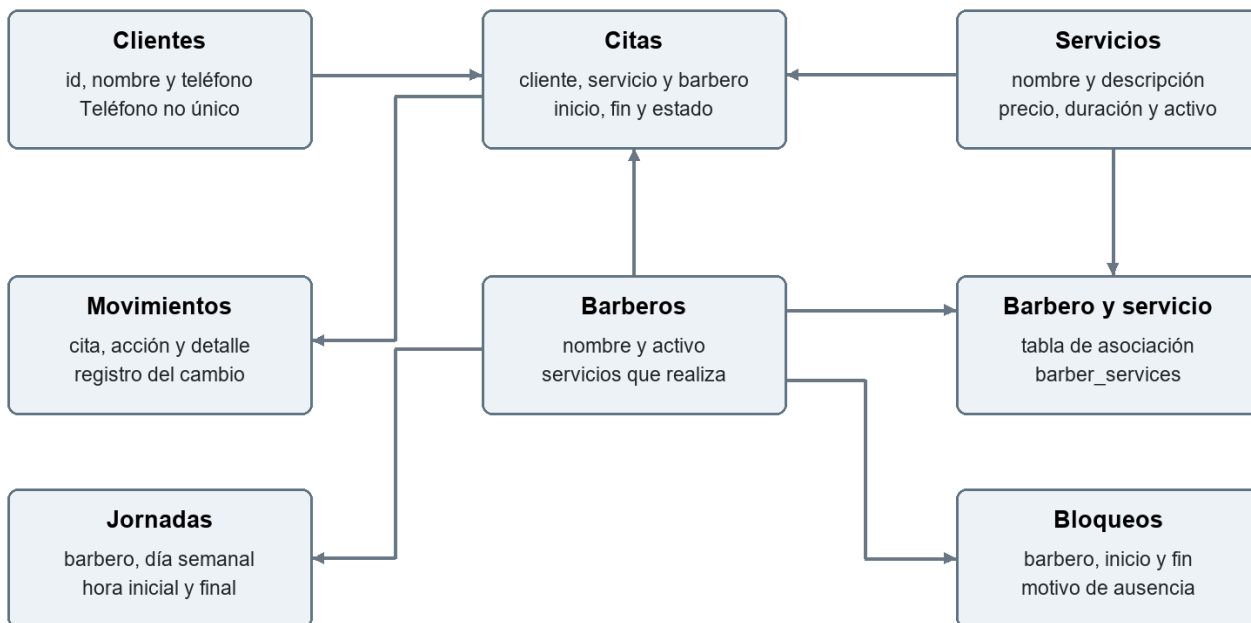


Figura 3. Relaciones de la base de datos. Las flechas indican qué registros dependen de otros. Por ejemplo, un cliente puede tener varias citas, y cada cita puede tener varios movimientos.

Decisiones de persistencia

Decidí conservar el inicio y el fin de cada cita tal como quedan al reservar. Si mañana cambia la duración de un corte, las citas que ya estaban apartadas mantienen su horario. También guardan el nombre y teléfono recibidos en ese momento. El teléfono puede repetirse, porque dos personas pueden compartirlo. Si coinciden exactamente nombre y teléfono, se reutiliza el cliente. Cuando dos personas comparten ambos datos, hace falta que el administrador revise el caso.

Una jornada puede dividirse en varios periodos. Por ejemplo, el espacio entre la salida de la mañana y la entrada de la tarde queda como descanso. Si los periodos se tocan, el sistema los toma como una jornada continua. Los bloqueos tienen fecha de inicio y fin, incluso para varios días. Si uno afecta una cita ya guardada, el panel lo avisa para que el encargado decida qué hacer. No la elimina automáticamente.

Otras tablas guardan administradores y sesiones (admins y admin_sessions), cambios de las citas (appointment_events), el bloqueo de agenda (schedule_mutex) y las actualizaciones de estructura (migrations). Los índices ayudan a consultar la agenda y buscar registros. Al iniciar, el sistema adapta la base anterior y vincula sus citas con clientes sin borrar la información existente.

Diseño de la interfaz

Para la pantalla mantuve el fondo oscuro y los detalles dorados de Mala Vida Barbers. Quise reunir en el mismo lugar lo que el cliente necesita para reservar: servicio, barbero, fecha y hora. Después escribe sus datos y confirma. La respuesta muestra el número de reserva y el horario que quedó apartado.

MALA VIDA BARBERS

Tu estilo. Tu momento.

Elige tu servicio y encuentra un espacio con tu barbero. Nosotros nos encargamos del resto.

Corte Clásico \$250.00 40 min
Corte y acabado a tu estilo.

Corte y Barba \$350.00 60 min
Corte completo y arreglo de barba.

Diseño de Barba \$150.00 30 min
Perfilado y cuidado de barba.

Sin cuenta, sin complicaciones. Tu reserva se confirma al terminar.

AGENDA TU VISITA

Reserva tu cita

Servicio: Corte Clásico - 40 min - \$250.00 Barbero: QA Barbero

Fecha: 13/10/2026

Horarios disponibles

10:00	10:10	10:20	10:30	10:40	10:50	11:00
11:10	11:20	11:30	11:40	11:50	12:00	12:10
12:20	12:30	12:40	12:50	13:00	13:10	13:20
15:00	15:10	15:20	15:30	15:40	15:50	16:00
16:10	16:20	16:30	16:40	16:50	17:00	17:10

Hora de Ciudad de México. Reservas con hasta 90 días de anticipación.

Nombre completo: Cliente Navegador Teléfono: 5598765432

Tus datos se usan para gestionar tu cita y solo son visibles para el administrador.

10:00 - Corte Clásico \$250.00

Confirmar reserva

MALA VIDA BARBERS Agenda con confianza. Llegas con estilo.

Figura 4. Pantalla de reserva durante una prueba automática. El nombre y el teléfono que aparecen son datos de prueba.

Administración y prevención de errores

En el panel, el encargado encuentra la agenda y los apartados de clientes, servicios, barberos, jornadas, bloqueos y cuenta. Puede buscar una cita por fecha, barbero, estado, nombre o teléfono. Antes de cancelar o cambiar un estado aparece una confirmación, para evitar hacerlo por un clic equivocado. El sistema también avisa si una jornada modificada o un bloqueo afecta citas que ya estaban guardadas.

En móvil, los formularios y las tarjetas quedan en una columna. La prueba con una pantalla de 390 por 844 píxeles no mostró desplazamiento hacia los lados. Los campos tienen etiquetas y se puede ver dónde está el foco del teclado. Los estados llevan texto, además del color. Eso cubre algunas comprobaciones técnicas, pero todavía falta ver cómo se maneja una persona que no conoce el sistema.

Desarrollo e implementación

Tecnologías y construcción

Usé React, Vite y Tailwind CSS para la interfaz. El servidor trabaja con JavaScript, Node.js y Express, y usa mysql2 para conectarse a MySQL. La base se conserva en un volumen de Docker, de modo que reiniciar no quite los datos. En la tabla agrupé las funciones que ya tiene el proyecto para mostrar qué quedó de cada parte.

Incremento funcional	Resultado implementado
Reservas y disponibilidad	Calcula horarios válidos y rechaza cruces, incluso ante solicitudes simultáneas.
Administración	Permite iniciar sesión, filtrar citas, cambiar su horario, cancelarlas y actualizar su estado.
Configuración	Permite asignar servicios a los barberos y definir sus jornadas y bloqueos.
Seguimiento y verificación	Conserva clientes e historial e incluye pruebas de la API y del navegador.

En la primera parte había propuesto cuatro iteraciones de dos semanas y herramientas como Trello, Figma y Visual Paradigm. Aquí puedo respaldar el trabajo con el código, los diagramas y las pruebas guardadas. No cuento con registros para asegurar las fechas de cada iteración o el uso de todas esas herramientas. Prefiero dejar esa diferencia clara entre lo planeado y lo que acompaña esta entrega.

Integración y conservación de datos

El inicio del sistema prepara las tablas que faltan y adapta la estructura anterior. También registra la actualización para no repetir la carga. Antes de guardar, el servidor revisa identificadores, fechas, teléfono y duración, y las consultas usan parámetros. Los cambios de agenda pasan por el mismo control de transacciones. Se probó volver a iniciar el sistema para comprobar que no aparecieran servicios ni jornadas duplicados.

Configuración de demostración

Para la demostración dejé la hora de Ciudad de México y una jornada inicial de lunes a sábado, de 10:00 a 14:00 y de 15:00 a 20:00. No todos los barberos tienen que quedarse con ese horario, porque se puede editar. El sistema permite reservar hasta 90 días antes y muestra opciones cada 10 minutos. Al agregar un barbero hace falta darle una jornada, y cada servicio debe asignarse a quienes lo realizan.

El proyecto se puede abrir en <http://127.0.0.1:3001> cuando está ejecutándose en el equipo. En el README están los pasos de instalación. Todavía queda publicar el repositorio remoto y poner su enlace en la entrega. Esa dirección local no permite que otra persona descargue el código.

Pruebas y resultados

Método de verificación

Para respaldar esta entrega se ejecutaron las pruebas el 23 de septiembre de 2026, en Windows. El entorno tenía Node.js 24.19.0, MySQL 8.0.45 y Microsoft Edge 153.0.4234.48. Se usaron datos ficticios y una base temporal, con solicitudes HTTP al servidor. Playwright 1.58.2 realizó el recorrido en el navegador. Al terminar se quitó la base de pruebas, sin borrar la del proyecto.

El resultado fue de cuatro casos básicos y 22 de integración aprobados. Son 26 casos. El registro muestra 27 porque suma también el resultado del grupo de integración completo. Lo aclaro para no contar ese grupo como si fuera otra prueba. La tabla reúne los resultados principales y los archivos del proyecto conservan los casos ejecutados.

Comprobación	Resultado observado
Solicitudes simultáneas	20 intentos sobre un mismo intervalo: 1 reserva creada y 19 rechazos por conflicto
Reprogramación inválida	El cambio se rechazó y la cita conservó su identificador y su hora de inicio.
Cancelación	La cita siguió en el historial y el horario volvió a estar libre.
Permisos	El servidor rechazó las consultas sin sesión con la respuesta HTTP 401.
Bloqueos y jornadas	El sistema excluyó los periodos ocupados y avisó de las citas afectadas.
Recorrido en navegador	Reserva, acceso, reprogramación, cancelación, historial y formularios administrativos completados
Resultado global	27 resultados aprobados, ninguno fallido ni omitido. Duración aproximada: 14.32 segundos.

Rendimiento observado

La medición fue pequeña: 20 consultas de disponibilidad, una tras otra, con conexión local y pocos datos. La mediana quedó en 16.5 ms y el máximo en 25.9 ms. En esas condiciones se cumplió la meta de menos de 2 segundos de RNF-04. Es el tiempo de respuesta de la API, no lo que tarda en cargar toda la página. Falta repetir la medición con más usuarios y datos.

Correcciones verificadas

Las pruebas ayudaron a corregir las etiquetas de los selectores y el manejo de periodos de trabajo consecutivos. También pasaron la compilación y el análisis estático de la interfaz, sin errores ni advertencias. Con esto puedo respaldar los casos revisados. Todavía puede aparecer un problema en un recorrido que no se haya probado.

Evaluación de calidad

Tomé ISO/IEC 25010:2023 como referencia general para ordenar la revisión de calidad [4]. En la tabla explico qué se comprobó y qué sigue pendiente. No hice una evaluación completa de la norma ni una certificación del sistema.

Aspecto	Criterio y evidencia	Valoración
Adecuación funcional	Las pruebas revisan las reservas y los cambios de las citas.	Las funciones principales están listas. La reserva pública confirma directamente.
Fiabilidad e integridad	Se probaron cruces, cambios rechazados, solicitudes simultáneas e historial.	Los casos aprobaron. Falta probar fallos prolongados.
Eficiencia	En 20 consultas locales: mediana de 16.5 ms y máximo de 25.9 ms.	Se cumplió la meta local de 2 s. Falta probar con mayor carga.
Seguridad	Exige sesión y usa script, opciones de protección de cookies y consultas con parámetros.	Se revisaron controles básicos. Falta una prueba de penetración.
Interacción y adaptación	Incluye etiquetas, avisos y confirmaciones. La vista móvil cabe en la pantalla.	La revisión técnica fue favorable. Falta probar con usuarios.
Mantenibilidad	El código separa módulos, comparte componentes y permite repetir pruebas.	La estructura facilita revisar el código. No se midió su cobertura.
Compatibilidad del entorno	Se usó Edge en la prueba final y Chrome durante el desarrollo.	Falta comprobar Firefox y Safari.

Métricas y alcance de los resultados

Pasaron los 26 casos, así que la aprobación de esa ejecución fue del 100 %. Eso no quiere decir que ya se haya probado todo el código. En la prueba de 20 intentos al mismo horario se guardó una sola reserva, que era lo esperado. También se revisó que las pantallas usadas en las pruebas cupieran en el ancho disponible.

Límites y siguientes validaciones

Antes de llevarlo a una barbería real, todavía tengo que revisar con el encargado cuánto tiempo antes se puede reservar y cómo va a manejar cancelaciones o retrasos. También hace falta probar con clientes, más registros y otros navegadores, y revisar mejor la accesibilidad. Si se publica, necesitará HTTPS, credenciales adecuadas y una forma de respaldar y recuperar los datos. Son pasos para continuar el proyecto.

Conclusiones y referencias

Conclusiones

En la primera parte dije que primero había que comprobar que el sistema reservara bien. Esta entrega ya permite revisar esa idea en un prototipo. Para ofrecer una hora, toma en cuenta la duración del servicio, la jornada y los descansos o bloqueos. Luego vuelve a comprobarlo al guardar. Esa segunda revisión es la que permite resolver el caso de dos personas que piden el mismo espacio.

Otra parte que quise conservar fue el historial. Si el encargado cancela o cambia una cita, puede consultar sus movimientos después. Si el cambio se rechaza, la cita anterior sigue ahí. La configuración por barbero también permite que los horarios ofrecidos correspondan a sus jornadas y a los servicios que realiza.

Por ahora cuento con un prototipo y con pruebas que respaldan la demostración académica. El paso que sigue es probarlo con un encargado y con clientes para saber qué resulta confuso o qué regla hace falta ajustar. Todavía no puedo decir que reduzca ausencias o mejore la atención, porque eso no se ha medido. Igual que en la primera entrega, dejo esas preguntas abiertas para revisarlas con un uso real.

Referencias

[1] INGE00031. (s. f.). Producto Transversal. Ingeniería de Software. Documento de la actividad, páginas 4 a 6.

[2] Oracle. (s. f.). MySQL 8.0 Reference Manual. Locking Reads. <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html>

[3] Node.js. (s. f.). Crypto. crypto.scrypt. <https://nodejs.org/api/crypto.html#cryptoscryptpassword-salt-keylen-options-callback>

[4] ISO e IEC. (2023). ISO/IEC 25010:2023. Systems and software engineering. Systems and software Quality Requirements and Evaluation. Product quality model. Ficha pública del estándar. <https://committee.iso.org/standard/78176.html>

Fuentes web consultadas el 20 de septiembre de 2026.

Uso de herramientas de apoyo

Usé inteligencia artificial como apoyo en la implementación y revisión del código, así como en la preparación y revisión del informe.

Para describir el funcionamiento tomé como base el código y las pruebas del proyecto. Aún no se han realizado entrevistas ni pruebas con clientes reales.